

Project 4: Kernel Hacking – documentation

Student: Lauri Metso

Repository: <https://github.com/laurimetso/systeemiohjelmointi>

Summary

This project modifies xv6 by adding a custom system call `getreadcount()`, which returns the number of times the kernel `read()` operation has been executed since boot. This implementation introduces a global variable in the kernel, increments it whenever `fileread()` is called, and exposes the value to user programs through a new system call. A small user-space test program was also created to verify functionality.

Setup

Clone xv6

git clone <https://github.com/mit-pdos/xv6-public.git>

Build and run xv6

make qemu

Implementation

Global read counter: a global variable was added to `sysfile.c`:

```
int readcount = 0;
```

This variable stores the total number of read operations performed by the kernel.

Incrementing the counter: In `file.c`, the counter is referenced with `extern int readcount;` and the counter is incremented inside `fileread()` with `readcount++`. Every call to the kernel read handler updates the count.

```
extern int readcount;
```

```
GNU nano 6.2 f
}
return -1;
}

// Read from file f.
int
fileread(struct file *f, char *addr, int n)
{
    int r;
    readcount++;
    if(f->readable == 0)
        return -1;
    if(f->type == FD_PIPE)
        return piperead(f->pipe, addr, n);
    if(f->type == FD_INODE){
        ilock(f->ip);
        if((r = readi(f->ip, addr, f->off, n)) > 0)
            f->off += r;
        iunlock(f->ip);
        return r;
    }
    panic("fileread");
}
```

System call implementation: In sysfile.c, implemented the system call handler:

```
int readcount = 0;

int
sys_getreadcount(void)
{
    return readcount;
}
```

Registering the system call: in syscall.h, added a syscall number:

```
#define SYS_getreadcount 22
```

In syscall.c, added the function declaration and registered the syscall:

```
extern int sys_getreadcount(void);
```

```
[SYS_getreadcount] sys_getreadcount,
};
```

User-space interface

In user.h, added the user-space function declaration:

```
int uptime(void);
int getreadcount(void);
// ulib.c
```

In usys.S added the syscall stub:

```
SYSCALL(getreadcount)
```

Test program

A small user-space test program test.c was created to verify the syscall.

```
GNU nano 6.2 test.c
#include "types.h" // xv6 type definitions
#include "user.h" // user space system call declaration

int
main(void)
{
    // store current number of read() calls before testing
    int before = getreadcount();

    // buffer to store data
    char buf[10];
    // open readme in read only
    int fd = open("README",0);

    // read 10 bytes twice into buf
    read(fd,buf,10);
    read(fd,buf,10);

    // store updated count
    int after = getreadcount();

    // print read count before and after testing
    printf(1, "before: %d\n", before);
    printf(1, "after: %d\n", after);
    exit();
}
```

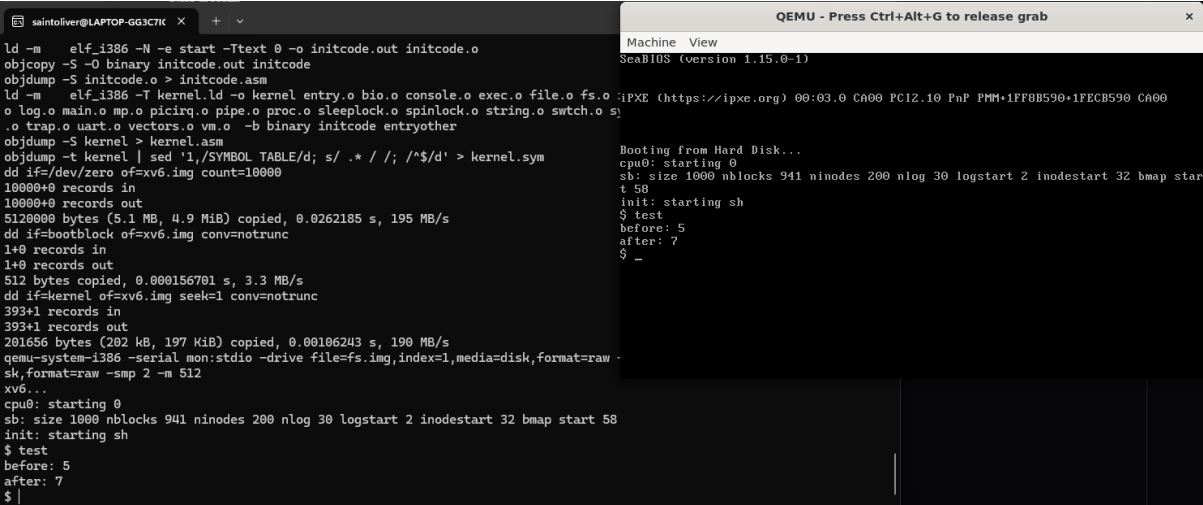
Makefile modification:

The test program was added to UPROGS in the Makefile (_test\):

```
UPROGS=\
    _cat\
    _echo\
    _forktest\
    _grep\
    _init\
    _kill\
    _ln\
    _ls\
    _mkdir\
    _rm\
    _sh\
    _stressfs\
    _usertests\
    _wc\
    _zombie\
    _test\
```

Rebuilding xv6:

After all modifications, I ran make clean and make qemu. Example usage:



Scored features

Feature	Score
Works	Required, 2
Documentation	Required, 1

Notes

readcount is global and persists for the lifetime of the kernel session. The counter resets when xv6 is rebooted.

The counter increment occurs in `fileread()` rather than the syscall wrapper itself, meaning all kernel file reads are counted.

The test program verifies that invoking `read()` updates the counter and that the value is accessible through the custom syscall.

Code was commented after taking screenshots.